Instrukcje warunkowe

Powtórka: wykład 1

- Projekt: swift package init, swift run, swift test
- let i var, typy, jawna konwersja
- T? i ?? dla brakującej wartości
- Funkcje z etykietami argumentów, struct
- Testy: @Test i #expect

Powtórka: warunki

- if, operator ?: i switch były w skrócie
- Dziś szczegóły

```
let age = Int(readLine() ?? "") ?? 0

if age >= 18 {
    print("adult")
} else {
    print("minor")
}

let label = age >= 18 ? "adult" : "minor"

switch age {
case 0..<18: print("minor")
default:     print("adult")
}
```

Plan wykładu

- Porównania i operatory logiczne
- if: kolejność gałęzi, granice, wyrażenia
- guard i if let z kilkoma warunkami
- switch: tekst, przedziały, krotki, where
- Typ wyliczeniowy enum
- Testy warunków

Operatory porównania

- = przypisuje, == porównuje
- Wynik porównania to Bool
- Napisy porównujemy alfabetycznie, wielkość liter ma znaczenie

```
let points = 74

points == 74              // true
points != 50              // true
points >= 75              // false

"cat" == "Cat"            // false
"cat" < "dog"             // true
```

Operatory logiczne

- && i: oba warunki prawdziwe
- || lub: wystarczy jeden
- ! nie: zaprzeczenie

```
let age = 20
let hasTicket = true

age >= 18 && hasTicket          // true
age < 7 || age > 65             // false
!hasTicket                      // false
```

Priorytet operatorów

- && wiąże mocniej niż ||
- Nawiasy i tak piszemy, dla czytelności

```
let year = 2028

let isLeapYear = (year % 4 == 0 && year % 100 != 0) || year % 400 == 0
```

Nazwany warunek

- Złożony warunek w stałej typu Bool
- Nazwa mówi, co sprawdzamy

```
let age = 16
let hasParentalConsent = true

let isAdult = age >= 18
let canEnter = isAdult || hasParentalConsent

if canEnter {
    print("Welcome")
}
```

Skrócone wartościowanie

- false && ... nie liczy prawej strony
- true || ... też nie
- Lewy warunek chroni prawy

```
let dividend = 12
let divisor = Int(readLine() ?? "") ?? 0

if divisor != 0 && dividend % divisor == 0 {
    print("Divisible")
} else {
    print("Not divisible or divisor is zero")
}
```

Kolejność gałęzi

- Wykona się tylko pierwsza pasująca gałąź
- Kolejny warunek nie musi powtarzać poprzedniego

```
let points = 78

if points >= 90 {
    print("very good")
} else if points >= 75 {
    print("good")
} else if points >= 50 {
    print("satisfactory")
} else {
    print("fail")
}
```

Zła kolejność

- Kompilator nie ostrzeże
- Wykryje to test dla każdej gałęzi

```
if points >= 50 {
    print("satisfactory")      // also catches 78 and 95
} else if points >= 75 {
    print("good")              // never runs
}
```

Granice przedziałów

- < czy <=: zawsze sprawdź wartość na granicy
- „Do 25” znaczy <= 25
- Testujemy granicę i wartości obok niej

```
let temperature = 25.0

if temperature < 0 {
    print("frost")
} else if temperature <= 25 {
    print("mild")                  // 0 and 25 land here
} else {
    print("heat")
}
```

Zagnieżdżanie a łączenie

```
// nested
if number > 0 {
    if number % 2 == 0 {
        print("positive even")
    }
}

// combined
if number > 0 && number % 2 == 0 {
    print("positive even")
}
```

Wczesny return

- return kończy funkcję, else nie jest potrzebne
- Każdy warunek na tym samym poziomie wcięcia

```
func describe(_ number: Int) -> String {
    if number == 0 {
        return "zero"
    }
    if number < 0 {
        return "negative"
    }
    return "positive"
}
```

if jako wyrażenie

- Od Swifta 5.9
- Każda gałąź to jedna wartość tego samego typu
- else jest obowiązkowe

```
let temperature = 31

let description = if temperature > 30 {
    "heat"
} else if temperature < 0 {
    "frost"
} else {
    "normal"
}
```

Oficjalny przykład: if

```
let individualScores = [75, 43, 103, 87, 12]
var teamScore = 0
for score in individualScores {
    if score > 50 {
        teamScore += 3
    } else {
        teamScore += 1
    }
}
print(teamScore)
// Prints "11".

let scoreDecoration = if teamScore > 10 {
    "🎉"
} else {
    ""
}
print("Score:", teamScore, scoreDecoration)
// Prints "Score: 11 🎉".
```

Reszta z dzielenia

- Znak reszty jest taki jak znak dzielnej
- Parzystość: % 2 == 0, nie % 2 == 1
- isMultiple(of:) czyta się łatwiej

```
7 % 2                          // 1
-7 % 2                         // -1
-7 % 2 == 1                    // false

12.isMultiple(of: 4)           // true
(-7).isMultiple(of: 2)         // false
```

Gotowe funkcje zamiast if

- max, min i abs są w bibliotece standardowej
- Przyjmują dowolnie wiele argumentów

```
let largest = max(3, 17, 8, 17)              // 17
let smallest = min(3, 17, 8)                 // 3
let distance = abs(-12)                      // 12

let percent = 130
let clampedPercent = min(max(percent, 0), 100)    // 100
```

guard ze zwykłym warunkiem

- guard działa nie tylko z let
- Najpierw odrzucamy złe dane, dalej główny kod

```
func quotient(_ dividend: Int, by divisor: Int) -> Int? {
    guard divisor != 0 else {
        return nil
    }
    return dividend / divisor
}

quotient(7, by: 2)          // Optional(3)
quotient(7, by: 0)          // nil
```

guard z kilkoma warunkami

- Przecinek działa jak &&
- Warunki sprawdzane po kolei

```
func isTriangle(_ a: Double, _ b: Double, _ c: Double) -> Bool {
    guard a > 0, b > 0, c > 0 else {
        return false
    }
    return a + b > c && a + c > b && b + c > a
}
```

if let z kilkoma warunkami

- Rozpakowanie i warunek w jednym if
- Kolejne warunki widzą stałe z poprzednich

```
if let dividend = Int(readLine() ?? ""),
   let divisor = Int(readLine() ?? ""),
   divisor != 0 {
    print("Quotient: \(dividend / divisor)")
} else {
    print("Invalid input or division by zero")
}
```

switch na tekście

- Tekstów jest nieskończenie wiele, więc default jest konieczny
- Kilka wartości w jednym case

```
let answer = readLine() ?? ""

switch answer.lowercased() {
case "y", "yes":
    print("Continuing")
case "n", "no":
    print("Done")
default:
    print("Unknown answer: \(answer)")
}
```

switch z przedziałami

- ..<b, a..<b, a...: przedziały jednostronne i półotwarte
- Działa dla Int i Double
- Pierwszy pasujący case wygrywa

```
let bmi = 23.4

switch bmi {
case ..<18.5:     print("underweight")
case 18.5..<25:   print("normal")
case 25..<30:     print("overweight")
default:          print("obese")
}
```

Pusty case

- Każdy case musi mieć co najmniej jedną instrukcję
- break: świadomie nic nie robimy

```
let errorCode = 0

switch errorCode {
case 0:
    break
case 1...9:
    print("Warning \(errorCode)")
default:
    print("Error \(errorCode)")
}
```

switch na krotce

- Kilka wartości dopasowanych naraz
- _ pasuje do każdej wartości
- W krotce można używać przedziałów

```
let point = (2, -1)

switch point {
case (0, 0):                print("origin")
case (_, 0):                print("on the x-axis")
case (0, _):                print("on the y-axis")
case (-2...2, -2...2):      print("near the origin")
default:                    print("far away")
}
```

let i where w case

- let wyciąga wartość do stałej
- where dodaje warunek
- Ostatni case łapie wszystko, default nie jest potrzebny

```
let point = (3, 3)

switch point {
case let (x, 0):
    print("on the x-axis, x = \(x)")
case let (x, y) where x == y:
    print("on the line y = x")
case let (x, y):
    print("point (\(x), \(y))")
}
```

Oficjalny przykład: switch

```
let vegetable = "red pepper"
switch vegetable {
case "celery":
    print("Add some raisins and make ants on a log.")
case "cucumber", "watercress":
    print("That would make a good tea sandwich.")
case let x where x.hasSuffix("pepper"):
    print("Is it a spicy \(x)?")
default:
    print("Everything tastes good in soup.")
}
// Prints "Is it a spicy red pepper?"
```

switch jako wyrażenie

- Od Swifta 5.9, tak jak if
- Każdy case to jedna wartość

```
let dayNumber = 6

let dayType = switch dayNumber {
case 1...5: "weekday"
case 6, 7: "weekend"
default: "no such day"
}
```

Typ wyliczeniowy enum

- Zamknięty zbiór możliwości
- Zamiast napisów i liczb-kodów
- Gdy typ jest znany, wystarczy kropka

```
enum TimeOfDay {
    case morning
    case noon
    case evening
}

var now = TimeOfDay.morning
now = .evening
```

enum w switch

- Kompilator zna wszystkie przypadki
- default nie jest potrzebny

```
switch now {
case .morning:
    print("Good morning")
case .noon:
    print("Enjoy your lunch")
case .evening:
    print("Good night")
}
```

enum z inicjalizatorem

- init wybiera przypadek na podstawie liczby
- Każda gałąź musi ustawić self

```
enum Grade: Equatable {
    case fail, pass, good

    init(points: Int) {
        switch points {
        case ..<50:     self = .fail
        case 50..<75:   self = .pass
        default:        self = .good
        }
    }
}
```

Wartość surowa

- Każdy przypadek ma przypisaną liczbę albo napis
- Kolejne liczby liczone automatycznie
- init(rawValue:) zwraca opcjonal

```
enum Weekday: Int {
    case monday = 1
    case tuesday, wednesday, thursday, friday, saturday, sunday
}

Weekday.wednesday.rawValue              // 3
let saturday = Weekday(rawValue: 6)     // Optional(.saturday)
let missing = Weekday(rawValue: 9)      // nil
```

CaseIterable

- allCases: wszystkie przypadki
- Dla String wartość surowa to nazwa przypadku

```
enum Color: String, CaseIterable {
    case red, green, blue
}

Color.allCases.count            // 3
Color.green.rawValue            // "green"
```

Wartości powiązane

- Przypadek może nieść dane
- Każdy przypadek inne dane

```
enum Payment {
    case cash
    case card(number: String)
    case installments(count: Int)
}

let payment = Payment.installments(count: 10)
```

Odczyt wartości powiązanych

- Dane wyciągamy w switch przez let
- Każdy przypadek obsłużony, default niepotrzebny

```
switch payment {
case .cash:
    print("cash")
case .card(let number):
    print("card \(number)")
case .installments(let count):
    print("\(count) installments")
}
```

Oficjalny przykład: enum

```
enum Suit {
    case spades, hearts, diamonds, clubs

    func simpleDescription() -> String {
        switch self {
        case .spades:
            return "spades"
        case .hearts:
            return "hearts"
        case .diamonds:
            return "diamonds"
        case .clubs:
            return "clubs"
        }
    }
}
let hearts = Suit.hearts
let heartsDescription = hearts.simpleDescription()
```

Właściwość obliczana

- Liczona przy każdym odczycie
- Wywołanie bez nawiasów, jak pole
- extension dopisuje kod do istniejącego typu

```
extension Grade {
    var isPassing: Bool {
        return self != .fail
    }
}

Grade(points: 74).isPassing       // true
Grade(points: 20).isPassing       // false
```

Testy warunków

- Wartość na granicy i tuż obok
- Co najmniej jeden test na gałąź

```
@Test func gradeBoundaries() {
    #expect(Grade(points: 49) == .fail)
    #expect(Grade(points: 50) == .pass)
    #expect(Grade(points: 74) == .pass)
    #expect(Grade(points: 75) == .good)
}
```

Testy sparametryzowane

- Ten sam test dla wielu danych
- Każdy argument w raporcie osobno

```
@Test(arguments: [0, 25, 49])
func failingPoints(points: Int) {
    #expect(Grade(points: points) == .fail)
}
```

Dane i oczekiwany wynik

- zip łączy dwie listy w pary
- Para: dane wejściowe i oczekiwany wynik

```
@Test(arguments: zip(
    [49, 50, 75, 100],
    [Grade.fail, .pass, .good, .good]
))
func grade(points: Int, expected: Grade) {
    #expect(Grade(points: points) == expected)
}
```

Program z warunkami

- Ten sam schemat co na wykładzie 1
- Logika w funkcji, main.swift tylko czyta i drukuje

```
public func runGrade(input: [String]) -> [String] {
    guard let points = Int(input.first ?? "") else {
        return ["Not an integer"]
    }
    switch Grade(points: points) {
    case .fail:
        return ["Grade: 2"]
    case .pass:
        return ["Grade: 3"]
    case .good:
        return ["Grade: 4"]
    }
}
```

Test integracyjny

- Każda gałąź programu, także złe dane

```
@Test func gradeProgram() {
    #expect(runGrade(input: ["74"]) == ["Grade: 3"])
    #expect(runGrade(input: ["90"]) == ["Grade: 4"])
    #expect(runGrade(input: ["abc"]) == ["Not an integer"])
    #expect(runGrade(input: []) == ["Not an integer"])
}
```

```
$ echo 74 | swift run -q Grades
Grade: 3
```

Podsumowanie

- Kolejność gałęzi i granice przedziałów decydują o wyniku
- guard odrzuca złe dane na początku funkcji
- switch musi być wyczerpujący
- enum zamiast napisów i liczb-kodów
- Testujemy każdą gałąź i każdą granicę

Materiały

- A Swift Tour, Control Flow: docs.swift.org/swift-book
- The Swift Programming Language: Basic Operators, Control Flow, Enumerations
- Swift Testing, testy sparametryzowane: developer.apple.com/documentation/testing

Źródła

- [1] The Swift Programming Language, A Swift Tour, The Swift Project. docs.swift.org/swift-book (dostęp: 26.10.2026)
- [2] The Swift Programming Language, Basic Operators, Control Flow, Enumerations. docs.swift.org/swift-book (dostęp: 26.10.2026)
- [3] Swift Testing, dokumentacja Apple Developer. developer.apple.com/documentation/testing (dostęp: 26.10.2026)